Grokking System Design Fundamentals

The Complete 18-Part Chapter-by-Chapter LinkedIn Content Series

Post #01: Scalability, Latency vs Throughput & High Availability SLAs

Target File: 01 System Design Fundamentals.html

```
■ Latency vs Throughput: The most common confusion in System Design Interviews:

Most engineers think making a system faster (lower latency) automatically increases throughput. But in distributed
systems, they are two completely different dimensions:

■■ Latency: The time taken to process a single request (e.g. 50ms P99 response time).
■ Throughput: The number of requests processed per second (e.g. 100,000 QPS).

■ The Classic Pipeline Tradeoff:
• Batching requests (e.g. waiting 10ms to group 100 events) increases Throughput by 5x, but slightly increases
individual Latency!
• Streaming requests one-by-one gives minimum Latency, but reduces overall Throughput due to network framing
overhead.

■ Availability "The 9's" Quick Sizing:
• 99.9% (Three 9s) = 8.76 hours downtime/year.
• 99.99% (Four 9s - Standard FAANG SLA) = 52.6 minutes downtime/year.
• 99.999% (Five 9s - Mission Critical) = 5.26 minutes downtime/year!

■ How do you measure latency in production: P90, P95, or P99?

(■ Full live interactive guide in first comment! ■)
#SystemDesign #Latency #Throughput #DistributedSystems #Performance #SoftwareEngineering
```

Post #02: Layer 4 vs Layer 7 Load Balancing & Weighted Round-Robin

Target File: 02 Load Balancing.html

```
■■ Layer 4 (Transport) vs Layer 7 (Application) Load Balancing: Which one should you pick?

Load balancers are the gatekeepers of scalable systems. Choosing the wrong tier can cause severe packet
bottlenecking:

■ Layer 4 Load Balancing (TCP/UDP):
• Operates at IP & Port level without decrypting TLS or reading HTTP payloads.
• Ultra-high throughput (millions of packets/sec via Linux IPVS / Maglev).
• Limitation: Cannot inspect HTTP headers, cookies, or route by URL path!

■ Layer 7 Load Balancing (HTTP/HTTPS/gRPC):
• Decrypts TLS and parses headers, cookies, and JSON payloads.
• Smart routing: `/api/v1/checkout` -> Payment Cluster; `/api/v1/search` -> Search Cluster.
• Features: Header-based rate limiting, JWT validation, WebSocket session affinity.
• Tradeoff: Higher CPU utilization due to SSL termination and packet parsing.

■ Production Best Practice:
A two-tier setup: Layer 4 ECMP/Maglev LB at the network edge for raw packet distribution -> Fans out to a fleet of
Layer 7 Envoy/NGINX gateways for application routing!

■ What load balancer do you run in production: Envoy, NGINX, HAProxy, or Cloud Load Balancers?

(■ Full live interactive guide in first comment! ■)
#LoadBalancing #Networking #DevOps #SystemDesign #Infrastructure #Microservices
```

Post #03: API Gateway Architecture: Rate Limiting, Auth & Circuit Breaking

Target File: 03 API Gateway.html

```
■ Why you should NEVER expose microservices directly to the public internet:

Without an API Gateway, every mobile and web client must talk directly to dozens of internal microservices, causing:
■ Tight coupling between client and internal service endpoints.
■ Duplicated authentication, SSL certificates, and logging logic in every service.
■ Vulnerability to DDoS attacks and noisy-neighbor resource starvation.
```

```
■ The API Gateway Pattern solves this by acting as a unified entry point:
1■■ Authentication & Authorization: Validates JWT / OAuth2 tokens at the edge and passes sanitized user identity headers (`X-User-Id: 42`) to downstream services.
2■■ Protocol Translation: Translates public REST/JSON requests into internal high-speed gRPC/Protobuf calls.
3■■ Rate Limiting & WAF: Enforces Token Bucket / Sliding Window limits per API key before traffic reaches backend workers.
4■■ SSL Termination & Circuit Breaking: Absorbs TLS encryption overhead and prevents cascading backend failures.

■ Do you use an off-the-shelf gateway (Kong, Envoy, AWS API Gateway) or build custom in Node/Go?

(■ Full live interactive guide in first comment! ■)
#APIGateway #Microservices #Security #Envoy #BackendEngineering #SystemDesign
```

Post #04: HTTP/1.1 vs HTTP/2 vs HTTP/3 (QUIC) vs WebSockets

Target File: 04 Networking Protocols.html

```
■ The evolution of web protocols: Why HTTP/3 moves from TCP to UDP (QUIC):

Understanding networking protocols is essential for optimizing client-server latency:

1■■ HTTP/1.1: Head-of-Line (HoL) Blocking
• Only 1 request/response per TCP connection at a time. Browsers open 6 parallel TCP connections to compensate, causing connection churn.

2■■ HTTP/2: Binary Framing & Multiplexing
• Multiplexes dozens of concurrent streams over a single TCP connection.
• Limitation: TCP-level packet loss stalls ALL multiplexed streams (TCP Head-of-Line blocking).

3■■ HTTP/3 over QUIC (UDP):
• Runs over UDP with user-space congestion control.
• Solves TCP HoL blocking: Packet loss on Stream A does NOT block Stream B!
• Zero-RTT Handshake: Re-establishes encrypted connections in 0 milliseconds using cached connection tokens.

4■■ WebSockets:
• Full-duplex persistent bidirectional TCP connection with minimal 2-byte framing overhead (ideal for multiplayer games, chat, Figma canvas sync).

■ Is your production API running HTTP/2 or HTTP/3 yet?

(■ Full live interactive guide in first comment! ■)
#Networking #HTTP3 #QUIC #WebSockets #Performance #SystemDesign
```

Post #05: DNS Anycast Routing, GeoDNS & Latency Optimization

Target File: 05 Domain Name System (DNS).html

```
■ How Geo-DNS & BGP Anycast route users to the closest data center in < 20ms:

When a user types `google.com`, how does the internet ensure they connect to a server 50 miles away rather than across the ocean?

■ Two Core Routing Mechanisms:

1■■ GeoDNS (Latency-Based Routing):
• The DNS resolver looks up the client's recursive DNS IP.
• Returns the IP address of the geographically closest data center.
• Limitation: Relies on recursive resolver location (which may differ from actual user IP if using 8.8.8.8 without EDNS-Client-Subnet).

2■■ BGP Anycast Routing (Cloudflare, AWS Route53):
• Multiple physical data centers across 300+ global cities advertise the EXACT SAME IP address via Border Gateway Protocol (BGP).
• Internet routers automatically steer user packets along the shortest physical AS-path to the nearest edge point of presence (PoP)!

■ What DNS provider do you rely on: AWS Route53, Cloudflare, or NS1?

(■ Full live interactive guide in first comment! ■)
#DNS #Networking #Anycast #Cloudflare #Infrastructure #SystemDesign
```

Post #06: Caching Strategies: Cache-Aside vs Write-Through vs Write-Behind

Target File: 06 Caching and CDN.html

```
■ Cache-Aside, Write-Through, Write-Behind: How to pick the right caching topology:

Caching is the #1 way to achieve sub-millisecond read latency, but picking the wrong write strategy can corrupt your
data!

1■■ Cache-Aside (Lazy Loading):
• Read: App queries Cache. On Miss, reads DB and populates Cache.
• Write: App writes to DB, then evicts/invalidates cache key.
• Best for: General read-heavy systems (e.g. user profiles).

2■■ Write-Through:
• App writes to Cache; Cache synchronously writes to DB before confirming.
• Strength: Cache is never stale.
• Weakness: Double-write latency penalty on every update.

3■■ Write-Behind (Write-Back):
• App writes to Cache; Cache immediately confirms and asynchronously batches disk flushes!
• Strength: Ultra-fast write throughput (absorbs write bursts).
• Risk: If the cache crashes before flushing to disk, recent writes are LOST!

4■■ Eviction Policies:
• LRU (Least Recently Used), LFU (Least Frequently Used), FIFO, and 2Q.

■ What is your go-to cache eviction policy in Redis?

(■ Full live interactive guide in first comment! ■)
#Caching #Redis #Performance #SystemDesign #SoftwareEngineering #Backend
```

Post #07: Database Sharding: Range vs Hash vs Directory-Based Partitioning

Target File: 07 Data Partitioning and Sharding.html

```
■■ When your database exceeds 5TB and 50,000 QPS: How to shard like a FAANG architect:

Vertical scaling hits a physical hardware ceiling. Horizontal sharding splits large datasets across multiple
independent database servers.

■ 3 Sharding Strategies Compared:

1■■ Range-Based Sharding:
• Keys partitioned by value ranges (e.g., User IDs 1-1M -> Shard 1; 1M-2M -> Shard 2).
• Strength: Easy range queries (`BETWEEN 100 AND 500`).
• Weakness: Severe Hotspotting! New active users all hammer the highest shard.

2■■ Hash-Based Sharding (Consistent Hashing):
• `Shard = Hash(UserID) % N_Shards`.
• Strength: Perfectly uniform data distribution.
• Weakness: Range queries require querying ALL shards (Scatter-Gather).

3■■ Directory-Based Sharding:
• A centralized lookup service maps entity IDs to specific shard locations.
• Strength: Flexible dynamic rebalancing and custom customer tenant routing.
• Weakness: Lookup service becomes a single point of failure (SPOF) if not heavily cached.

■ What sharding key do you pick for multi-tenant SaaS: OrganizationId or UserId?

(■ Full live interactive guide in first comment! ■)
#Sharding #Databases #PostgreSQL #Scalability #SystemDesign #Architecture
```

Post #08: Forward Proxy vs Reverse Proxy vs VPN: The Structural Differences

Target File: 08 Proxies and VPN.html

```
■■ Forward Proxy vs Reverse Proxy vs VPN: Clear architectural breakdown:

Engineers often mix up Proxies and VPNs. Here is the definitive difference:

1■■ Forward Proxy (Protects the Client):
```

```
• Sits in front of client devices (e.g. corporate office).
• Intercepts outbound traffic to block malicious sites, cache external content, and mask employee IP addresses.
• Server sees the Proxy's IP, NOT the client's.

2■■ Reverse Proxy (Protects the Server):
• Sits in front of backend servers (e.g. NGINX, HAProxy).
• Intercepts inbound traffic from the internet to load balance, terminate SSL, and cache responses.
• Client sees the Proxy's IP, NOT the internal backend servers!

3■■ Virtual Private Network (VPN):
• Encrypts ALL device network traffic at the OS network interface level (Layer 3/4) through a secure tunnel
(WireGuard/IPSec) into a private network.

■ Do you use NGINX, Envoy, or Caddy as your reverse proxy of choice?

(■ Full live interactive guide in first comment! ■)
#Networking #Security #NGINX #Proxy #DevOps #SystemDesign
```

Post #09: Database Replication: Active-Passive vs Active-Active & Replication Lag

Target File: 09 Redundancy and Replication.html

```
■ Master-Replica vs Multi-Master Replication: Mitigating the dreaded Replication Lag:

Replication ensures data durability and high availability, but async replication introduces consistency traps:

1■■ Active-Passive (Primary-Replica):
• 1 Master handles all Writes. Multiple Replicas handle Reads.
• Synchronous Replication: Master waits for replica confirmation (zero data loss, but slow writes).
• Asynchronous Replication: Master commits locally immediately (ultra-fast writes, but risk of data loss on
failover).

2■■ Active-Active (Multi-Master):
• Multiple nodes accept both Reads and Writes.
• Requires Conflict Resolution (Last-Write-Wins timestamps, Vector Clocks, or CRDTs).

■■ The "Read-Your-Own-Writes" Consistency Trap:
User updates their profile picture (writes to Master) -> Immediately refreshes feed (reads from lagged Replica) ->
Old picture is shown!
■ Solution: Route reads for the updating user to Master for 5 seconds, or verify replica replication offset before
serving!

■ How do you handle read-after-write consistency in your systems?

(■ Full live interactive guide in first comment! ■)
#Databases #Replication #PostgreSQL #HighAvailability #SystemDesign
```

Post #10: CAP Theorem & PACELC: The Mathematical Tradeoffs of Distributed Systems

Target File: 10 CAP Theorem and Tradeoffs.html

```
■■ The mathematical reality of distributed systems: Why you can never have CA over wide-area networks:

Network cables get cut, routers reboot, and datacenters experience packet loss. In a distributed network, Network
Partition (P) is an unavoidable physical reality!

■ Therefore, your choice during a partition is strictly binary:

■ CP (Consistency + Partition Tolerance):
• Reject writes if quorum cannot be reached to prevent stale reads.
• Examples: Google Spanner, HBase, ZooKeeper, etcd.
• Use case: Banking balances, seat reservations, consensus state.

■ AP (Availability + Partition Tolerance):
• Accept writes on all reachable nodes, allowing divergent states to be merged later.
• Examples: Amazon DynamoDB, Apache Cassandra, CouchDB.
• Use case: Shopping carts, social media likes, telemetry logs.

■ PACELC Extension:
Even under normal operation (No partition), choose Latency (EL) or Consistency (EC)!
```

```
■ In your current architecture, did you pick CP or AP?

(■ Full live interactive guide in first comment! ■)
#CAPTheorem #DistributedSystems #SoftwareEngineering #Databases #SystemDesign
```

Post #11: B+ Trees vs LSM-Trees: How Database Storage Engines Work Under the Hood

Target File: 11 Databases and Indexing.html

```
■ B+ Tree vs LSM-Tree: Why PostgreSQL uses B+ Trees while Cassandra/RocksDB uses LSM-Trees:

The fundamental tradeoff in database internals is Random Read Latency vs Sequential Write Throughput:

■ B+ Trees (RDBMS: PostgreSQL, MySQL InnoDB):
• Ordered balanced tree where all data pointers live in leaf pages linked horizontally.
• Strength: Ultra-fast point reads ($O(\log N)$) and range scans.
• Bottleneck: Writes require random disk I/O to update tree pages, causing write amplification.

■ LSM-Trees (Log-Structured Merge-Trees: Cassandra, RocksDB, ScyllaDB):
• Writes append sequentially to an in-memory `MemTable` (SkipList) and Write-Ahead Log (WAL).
• When full, flushes to disk as immutable `SSTables`.
• Background compaction merges SSTables and purges tombstones.
• Strength: Blazing-fast sequential write throughput ($O(1)$ append).
• Tradeoff: Reads must check MemTable + Bloom Filters + multiple SSTables.

■ If you are designing an IoT sensor ingestion platform, which storage engine do you choose?

(■ Full live interactive guide in first comment! ■)
#Databases #PostgreSQL #Cassandra #DataStructures #SystemDesign #SoftwareArchitecture
```

Post #12: Bloom Filters: $O(1)$ Probabilistic Space-Saving Data Structures

Target File: 12 Bloom Filters.html

```
■ How Bloom Filters save Bigtable, Cassandra, and Redis from wasted disk lookups in $O(1)$ time:

How do you check if a username exists across 500,000,000 records without touching disk?

■ The Solution: Bloom Filters (A space-efficient probabilistic bit array).

1■■ The Mechanism:
• A bit array of size $m$ initialized to all 0s.
• Uses $k$ independent cryptographic hash functions.
• When adding an element: Set bits at indices $h_1(x), h_2(x), \dots, h_k(x)$ to 1.
• When querying: If ANY of the $k$ bits is 0 -> The element is **DEFINITIVELY NOT** in the set (0% False Negative)!
• If ALL $k$ bits are 1 -> The element is **PROBABLY** in the set (small, tunable False Positive probability).

2■■ The Math:
For 1 billion keys with 1% false positive rate: Requires only ~1.2 GB of RAM (less than 10 bits per key)!

3■■ Applications:
• Cassandra/RocksDB: Skips disk SSTable reads for non-existent keys.
• Web Crawlers: Avoids re-crawling URLs.
• Medium/Twitter: Avoids showing users recommended articles they already read.

■ Have you used Bloom Filters or Cuckoo Filters in your projects?

(■ Full live interactive guide in first comment! ■)
#BloomFilters #Algorithms #Redis #DataStructures #SystemDesign #Performance
```

Post #13: Distributed System Patterns: Heartbeats, Fencing Tokens & Write-Ahead Logs

Target File: 13 Distributed System Patterns.html

```
■■ 4 Battle-Tested Patterns for Building Bulletproof Distributed Systems:

Distributed systems are prone to partial failures, network splits, and GC pauses. Here are the 4 fundamental patterns
to survive them:

1■■ Write-Ahead Log (WAL):
• Append every transaction to an immutable disk log BEFORE updating in-memory state.
• Guarantees crash recovery and durability across sudden power outages.
```

```
2■■ Fencing Tokens (Preventing Split-Brain):
• Distributed locks can expire during long GC pauses.
• A monotonic incrementing fencing token ($T_1, T_2, T_3$) is issued with each lock.
• Storage rejects writes with older token numbers ($T < T_{current}$), preventing stale zombie masters from corrupting state!

3■■ Heartbeat with Phi Accrual Failure Detector:
• Instead of a binary up/down threshold, calculates a continuous suspicion probability based on historical network latency variance.

4■■ Idempotent Consumers:
• Every message carries a unique ID; consumers store processed IDs in Redis with atomic `SETNX`.

■ How do you handle zombie master split-brain in your clusters?

(■ Full live interactive guide in first comment! ■)
#DistributedSystems #Architecture #Reliability #DevOps #SystemDesign #Backend
```

Post #14: Zero Trust Security, TLS 1.3 Handshake & JWT Best Practices

Target File: 14 Security and Privacy.html

```
■ Zero-Trust Architecture: Why perimeter firewalls are dead in modern cloud engineering:

The old security model assumed everything inside the corporate intranet was safe. Zero Trust assumes the internal network is ALREADY compromised!

■ Core Zero-Trust Principles:

1■■ Mutual TLS (mTLS) Everywhere:
• Microservices don't just verify client identity; every East-West microservice connection validates cryptographic X.509 certificates in both directions via Envoy sidecars.

2■■ TLS 1.3 1-RTT Handshake:
• Slashes connection latency by removing round trips: Establishes cipher negotiation and key exchange in a single round trip (or 0-RTT resumption).

3■■ Stateless Short-Lived JWT + Refresh Token Rotation:
• Access tokens expire in 15 minutes (cryptographically verified locally via RS256 public key without hitting Auth DB).
• Refresh tokens stored in Redis with one-time-use rotation (any reused refresh token revokes the entire user session immediately!).

■ Do you use asymmetric RS256 or symmetric HS256 for your JWT signatures?

(■ Full live interactive guide in first comment! ■)
#Security #ZeroTrust #JWT #OAuth #DevOps #SystemDesign #CyberSecurity
```

Post #15: Message Brokers: Apache Kafka vs RabbitMQ vs AWS SQS

Target File: 15 Messaging Systems and Queues.html

```
■ Kafka vs RabbitMQ vs SQS: How to choose the right messaging backbone for your architecture:

Picking the wrong message broker can cause unmanageable queue backlogs or lost messages.

1■■ RabbitMQ (Smart Broker, Dumb Consumer):
• Routing: Rich AMQP routing exchanges (Direct, Fanout, Topic, Headers).
• Message Life: Deletes messages immediately upon consumer acknowledgment.
• Best for: Complex transactional routing, background tasks (Celery), and low-latency priority queues.

2■■ Apache Kafka (Dumb Broker, Smart Consumer):
• Model: Distributed, partitioned, immutable append-only commit log.
• Retention: Retains messages on disk for days/weeks. Consumers track their own read offsets.
• Replayability: Allows replaying historical event streams from any timestamp!
• Best for: High-throughput event streaming (1M+ events/sec), Change Data Capture (CDC), and real-time analytics.

3■■ AWS SQS:
• Model: Fully managed serverless queue.
• Best for: Cloud-native async task decoupling with zero infrastructure maintenance.
```

```
■ What is your primary queue technology: Kafka, RabbitMQ, SQS, or Redis Streams?

(■ Full live interactive guide in first comment! ■)
#ApacheKafka #RabbitMQ #MessagingQueues #EventDriven #SystemDesign #Backend
```

Post #16: Distributed Storage: Object Storage (S3) vs Block Storage (EBS) vs File Storage (NFS)

Target File: 16 Distributed File Systems.html

```
■ Object Storage (S3) vs Block Storage (EBS) vs File Storage (EFS): Structural differences:

Storage architecture dictates performance, latency, and cost at petabyte scale:

1■■ Block Storage (AWS EBS, SAN):
• Exposes raw storage blocks over high-speed networks (iSCSI, NVMe-oF).
• Can be formatted with any filesystem (ext4, XFS).
• Best for: Databases (PostgreSQL, MySQL) requiring sub-millisecond random I/O.
• Limitation: Typically single-instance attachment.

2■■ File Storage (NFS, AWS EFS):
• Hierarchical folder/file tree with POSIX permissions.
• Multi-attach: Hundreds of compute instances can mount and read/write concurrently.
• Best for: Shared application assets, legacy migrations, CMS file trees.

3■■ Object Storage (AWS S3, Ceph, MinIO):
• Flat namespace: Buckets and Keys with metadata headers.
• Accessible via REST APIs (`GET`, `PUT`, `DELETE`).
• Infinite horizontal scalability, 11 9's durability ($99.999999999\%$ via Reed-Solomon Erasure Coding).
• Best for: Media blobs, backups, video streaming, data lakes.

■ How much storage do your production applications manage: Gigabytes, Terabytes, or Petabytes?

(■ Full live interactive guide in first comment! ■)
#CloudStorage #AWS #S3 #DistributedSystems #DevOps #SystemDesign
```

Post #17: Monolith vs Microservices vs Serverless: The Evolutionary Roadmap

Target File: 17 System Architecture Patterns.html

```
■■ Monolith vs Microservices vs Serverless: When to evolve your architecture:

Premature microservices adoption is the #1 killer of startup velocity. Here is the realistic evolutionary roadmap:

1■■ The Majestic Modular Monolith (Phase 1):
• Single deployable unit with strictly enforced domain boundaries (folders/modules).
• Single ACID database. Zero distributed transaction overhead.
• Best for: Startups, new products, small engineering teams (< 20 engineers).

2■■ Microservices (Phase 2):
• Decouples domain services with private databases when team size exceeds 50+ engineers to prevent merge conflicts
and deployment gridlocks.
• Requires investment in: Service Mesh, Distributed Tracing (OpenTelemetry), CI/CD pipelines, and Kafka event buses.

3■■ Serverless & Edge Compute (Phase 3):
• Event-driven ephemeral execution (AWS Lambda, Cloudflare Workers).
• Best for: Spiky unpredictable traffic, image processing, webhook handlers, and edge personalization.

■ Is your team working on a Monolith or Microservices architecture?

(■ Full live interactive guide in first comment! ■)
#Architecture #Microservices #Monolith #Serverless #DevOps #SoftwareEngineering
```

Post #18: AI & LLM System Fundamentals: PagedAttention, FlashAttention & KV-Cache

Target File: 18 AI & Large Language Model (LLM) System Fundamentals.html

```
■ The Hardware & System Architecture of LLM Inference (FlashAttention & PagedAttention):

Why is LLM generation so computationally expensive, and how do modern serving engines optimize it?

■ 3 Breakthrough Systems Innovations:
```

1■■ KV-Cache & Memory Bottleneck:
In Autoregressive LLM generation, past Key and Value attention tensors are cached in GPU VRAM to avoid recalculating all tokens. At batch size 64 with 4k context: KV-Cache consumes 30GB+ of VRAM!

2■■ PagedAttention (vLLM):
Traditional allocators reserve contiguous VRAM for maximum context length, wasting 60-80% of GPU memory (internal fragmentation). PagedAttention manages KV-cache using virtual memory pages, boosting throughput by 4x!

3■■ FlashAttention-3:
Tiling the Attention matrix to fit entirely into fast on-chip GPU SRAM (100 TB/s bandwidth), avoiding slow High-Bandwidth Memory (HBM) round-trips!

4■■ Speculative Decoding:
A small 1B draft model generates 4 candidate tokens; a large 70B target model validates them in a single forward pass, yielding a 2.5x to 3x latency speedup!

■ Have you optimized LLM inference in production using vLLM or TensorRT-LLM?

(■ Full live interactive guide in first comment! ■)
#GenerativeAI #LLM #FlashAttention #vLLM #MachineLearning #SystemDesign